UTPORTO
Universidade Técnica do Porto

NORTE 2030
Programa Regional do Norte
PORTUGAL 2030
Cofinanciado pela União Europeia

~/tweb/aula-07

Git and GitHub_

Lesson 7 · 26 Oct 2026

CTeSP DTAM
Tecnologias Web

Lecturer
Gonçalo Reais

// review

Last lesson in brief

- **Global attributes**, such as `id`, `class`, `style` and `lang`, can be used on any element.
- An `id` identifies a single element on the page; a `class` groups several elements.
- `<audio>` and `<video>` play media with `controls`; a video with speech should have captions, with `<track>`.
- `<iframe>` shows content from another site, such as a video or a map, and needs a `title`.
- An **event**, such as a click, can run code; for an action, use a `<button>`, which also works with the keyboard.

// objectives

Lesson objectives

01

Explain what **version control** is for in a web project.

02

Record changes with `git init`, `git status`, `git add` and `git commit`.

03

Write useful **commit messages** that say what changed.

04

Put the project on **GitHub** and send new commits with `git push`.

// why versions

Versions in file names

Saving copies with different names does not say **what changed**, when, or why.

A **version control system**, such as Git, records every change to the project with that information, without loose copies.

tweb-project/

```
tweb-project/
├── site.html
├── site-final.html
├── site-final-v2.html
├── site-final-v2-fixed.html
└── site-final-FOR-REAL.html
```

// history

Where Git comes from

Git was created in 2005 by **Linus Torvalds**, the creator of Linux. The Linux team lost free access to the program it used to manage the versions of its code, and Torvalds wrote a replacement, whose first version was ready in a few days.

2005
Linus Torvalds creates Git, to manage the Linux code

2008
GitHub opens, a service for keeping repositories online

today
Git is the most widely used version control system

More in the short history of Git, in the Pro Git book.

// history

Why Git is used everywhere

Before Git, systems such as CVS and Subversion kept the history on a **central server**. In Git, every copy of the repository has the **full history** and works without the Internet.

standard

over 90% of developers use Git (Stack Overflow survey, 2022)

beyond code

it works for any project made of text files, such as websites and documentation

open source

the code of Linux and VS Code is in public Git repositories

// git

What Git is

Git is a program that records the **history** of a project. Each version has a code, the author, the date, a sentence that says what changed and the files changed, with the lines added (+) and removed (−). The folder keeps only the current version.

01 — 13 Oct 2026, 15:12

4e1b09a · Ana Silva

Add the about me page

`index.html` new, +38
`foto.jpg` new

02 — 19 Oct 2026, 15:47

9d2c8f1 · Ana Silva

Add contact form

`contacto.html` new, +41

03 — 20 Oct 2026, 15:31

c07a5e3 · Ana Silva

Fix the alt text of the photo

`index.html` +1 −1

04 — 26 Oct 2026, 14:58

f3b81d6 · current

Add link to the contact page in the nav

`index.html` +4
`contacto.html` +4

// installation

Installing on Windows

Git can be installed with the installer or from the terminal. At the end, close and reopen VS Code and check the installation with `git --version`.

With the installer

01 Download the installer from the Git for Windows install page.

02 Open the installer and accept the default options, with **Next**.

03 On the editor screen, choose **Use Visual Studio Code as Git's default editor**.

In the terminal

The same command works in PowerShell and in the VS Code terminal, which is the same terminal, inside the editor.

```
1  winget install --id Git.Git -e
```

> TIP On macOS, `git --version` in Terminal offers to install Git.

// terminal

Moving between folders in the terminal

The terminal is always **inside a folder**, which shows at the start of the line. Git commands apply to that folder.

COMMAND	WHAT IT IS FOR
`pwd`	shows the current folder
`ls`	lists what is in the folder
`cd meu-site`	goes into a folder
`cd ..`	goes back to the folder above

The `Tab` key completes the folder name.

```
PS C:\Users\ana> cd Documents
PS C:\Users\ana\Documents> ls
Mode    LastWriteTime     Name
d----   26/10/2026 14:02  meu-site
PS C:\Users\ana\Documents> cd meu-site
PS C:\Users\ana\Documents\meu-site>
```

On Windows, open **PowerShell** from the Start menu by typing "PowerShell". In VS Code, the terminal (`Ctrl` + `` ` ``) already is PowerShell and opens in the folder open in the editor.

// setup

Setting up Git

Once installed, Git needs a first-time setup, done once on each computer, with the name and email of the **author**. Without it, `git commit` fails with the message "Author identity unknown".

```
git --version
git config --global user.name "Ana Silva"
git config --global user.email "ana.silva@example.com"
git config --global init.defaultBranch main
```

The email should be the one used for the GitHub account. The last line names the history `main`, as GitHub does.

> **WARNING** On a classroom computer, set this up without `--global`, inside the repository, after `git init`. At the end of the lesson, sign out of GitHub in VS Code.

// tools

How to use Git

Git can be used with **commands** in the terminal or with **graphical programs**, which show buttons. They all work on the same repository.

```
> git status
> git add .
> git commit -m "…"
```

terminal

commands typed on the command line (CLI)

```
GitHub Desktop
GitKraken
Sourcetree
```

dedicated software

programs made for Git, with the history drawn on screen

```
Source Control
Ctrl+Shift+G
+  Commit  U  M
```

VS Code

the Source Control panel, already built into the editor

// under the hood

The .git folder

`git init` creates, inside the project folder, a hidden folder called `.git`, where Git keeps the whole history. Do not touch it, because deleting it deletes the history.

meu-site/

```
meu-site/
├── index.html
├── contacto.html
├── foto.jpg
└── .git/
    ├── objects/
    ├── refs/heads/main
    └── …
```

.git/

the hidden folder with the whole history

objects/

the commits and the files, each with its own hash

main

points to the latest commit and moves on with each one

// concepts

Three Git concepts

Git is built on three concepts, which come up throughout the lesson: the **repository**, the **history** and the **commit**.

```
meu-site/
├── index.html
├── contacto.html
└── .git/
```

repository

the project folder, with the history kept in the `.git` folder



history

the sequence of all the commits, each linked to the previous one

```
f3b81d6
Ana Silva
26 Oct 2026, 14:58
Add link to the contact page
in the nav
```

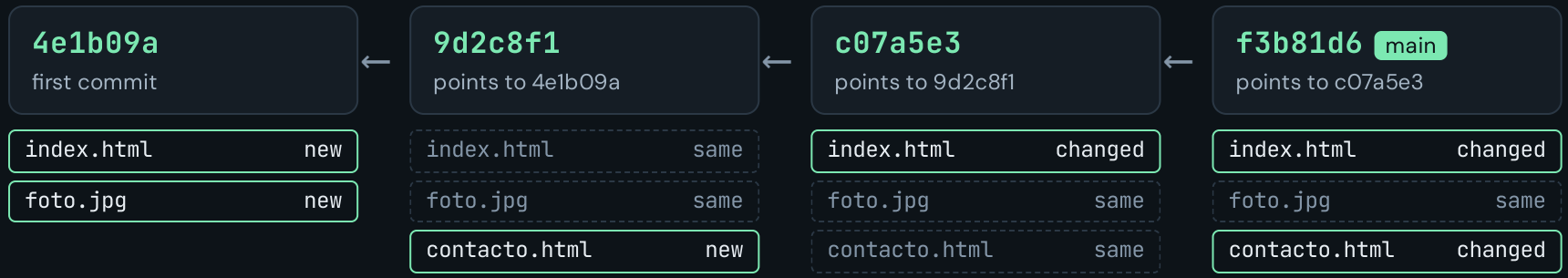
commit

a snapshot of the whole project at one moment, with author, date and message

// under the hood

How Git stores the history

Each commit stores a **full snapshot** of the files and points to the **previous** commit, forming a chain. A file that has not changed is not stored again, because the new commit reuses the previous version.



// commits

When and how to commit

When

- When a small change is **done and works**, such as fixing an `alt`.
- Before trying something that might go wrong.
- At the end of each work session.

Not after every line, and not once a week.

How

01 Edit and save the files.

02 Choose the changes with `git add`.

03 Record them with `git commit -m "…"`, with a message that says what changed.

The next slide shows what happens at each step.

// three areas

The three areas of Git



`git add` chooses the changes in the working folder that will go into the next commit and puts them in **staging**.

`git commit` saves everything in staging to the **repository**, as a new commit.

// three areas

A commit step by step

Ana fixed the `alt` in `index.html` and started changing `contacto.html`, which is not ready yet. With staging, only the `alt` fix goes into the commit.

STEP	WORKING FOLDER	STAGING	REPOSITORY
01 edit and save	`index.html` changed `contacto.html` changed	empty	`9d2c8f1`
02 `git add index.html`	`contacto.html` changed	`index.html`	`9d2c8f1`
03 `git commit -m "…"`	`contacto.html` changed	empty	`9d2c8f1` `c07a5e3` Fix the alt new

// commit messages

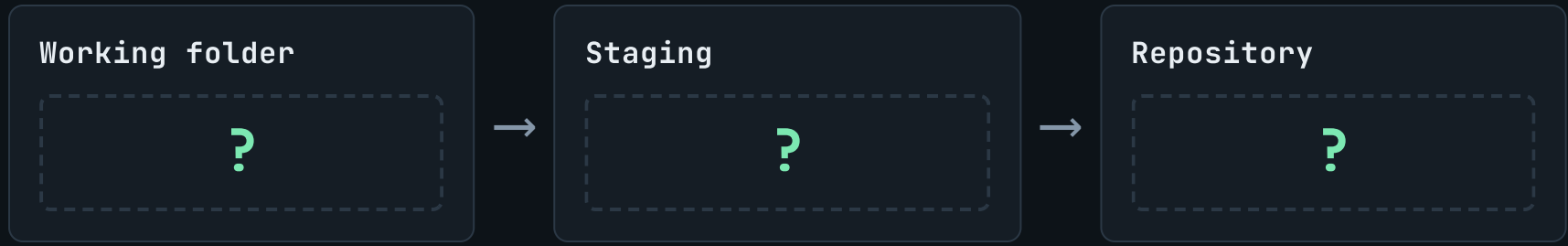
Commit messages

WRONG	`changes`	does not say what changed
WRONG	`asdf`	has no meaning
WRONG	`final final`	the file name problem again
RIGHT	`Add contact form`	says what changes in the site

Rule: start with a **verb** (Add, Fix, Change), say **what changes** in the site and make only one change per commit.

// git status

Which area is each change in?



git status answers this question. It tells you what has changed and is still only in the working folder, what has already been chosen for staging, and whether everything is saved in the repository. It changes nothing, it only shows.

// git status

Reading git status

```
Untracked files:
    contacto.html
    index.html
```

```
Changes to be committed:
    new file:   contacto.html
    new file:   index.html
```

```
nothing to commit, working tree clean
```

Untracked files

new files, only in the working folder

Changes to be committed

changes in staging, waiting for the commit

nothing to commit

everything saved in the repository

// commands

Commands learned so far

COMMAND	WHAT IT IS FOR
`git --version`	checks the installation
`git config --global`	sets your name and email
`git init`	creates the repository

COMMAND	WHAT IT IS FOR
`git status`	shows the state
`git add index.html`	one file to staging
`git add .`	everything to staging
`git commit -m "…"`	records a commit

Before each command, check in the terminal that you are in the **project folder**.

01 · activity

Activity 1

The first commit

23 min · individual

// activity 1

The first commit

01 In VS Code, open the `meu-site` folder, with `index.html` and `contacto.html` (**File → Open Folder**).

02 Open the terminal with `Ctrl` + `` ` `` and type `git init`, then `git status`.

03 Type `git add .` and then `git status` again.

04 Type `git commit -m "Add the first version of the site"`.

05 Type `git status` and compare it with the previous results.

> COMMON MISTAKE The message `fatal: not a git repository` means that the terminal is not in the project folder.

// history

The project’s history

↑ most recent

26 Oct	Add link to the contact page in the nav
20 Oct	Fix the alt text of the photo
19 Oct	Add contact form
13 Oct	Add the about me page

oldest

The **history** is the record of everything that was saved in the project, like a diary in which each entry is a commit. It reads from the most recent, at the top, to the oldest.

It is used to:

- know what changed and when;
- know who made each change;
- find the moment something stopped working.

// history

Viewing the history

`git log --oneline` shows one commit per line, from the **most recent**, at the top, to the oldest.

```
a3f9c21 (HEAD -> main) Add link to the contact page in the nav
7b2e0d4 Fix the alt text of the photo
1c84f6e Add the first version of the site
```

a3f9c21

the hash, a unique code that identifies the commit

(HEAD → main)

marks the commit the project is at now

message

the text written after -m

// differences

What changed between two versions

A **difference** compares two versions of the same file and shows only what changed, that is, what was removed and what was added.

previous version

I like photography and music.

new version

I like photography, music and cinema.

what changed

I like photography ~~and music~~, music and cinema.

The crossed-out text was removed and the underlined text was added. It is the same as track changes in a word processor.

// differences

Seeing what changed

`git diff` shows the lines changed since the last commit. Use it **before** `git add`, to check what will go into the commit.

- A line starting with `-` was **removed**.
- A line starting with `+` was **added**.
- Lines with no sign did not change and give context.

```
diff --git a/index.html b/index.html
--- a/index.html
+++ b/index.html
@@ -9,3 +9,3 @@
    <h1>About me</h1>
-   <img src="foto.jpg" alt="photo">
+   <img src="foto.jpg" alt="Portrait of Ana">
    <p>CTeSP DTAM student.</p>
```

// commands

New commands

They join the commands from activity 1.

COMMAND	WHAT IT IS FOR
`git log --oneline`	shows the history, one commit per line
`git diff`	shows what changed since the last commit

For each change in activity 2, the cycle is **edit and save**, `git diff`, `git add` and `git commit`.

> **TIP** If the result is long, the terminal shows it in parts. To leave, press the `q` key.

02 · activity

Activity 2

Three changes, three commits

15 min · individual

// activity 2

Three changes, three commits

01 Make a small change to the site, for example fix an `alt` in `index.html`, and save.

02 Look at the change with `git diff`.

03 Record it with `git add .` and `git commit -m "…"`, with a message that says what changed.

04 Repeat steps 1 to 3 for two more changes, such as a link in the `<nav>` to `contacto.html` and a new heading.

05 Check with `git log --oneline` that the history has at least 3 commits.

> Finished? Run `git status` after each step and read the state.

// interfaces

Buttons that hide commands

There are programs made for managing Git, and most code editors already have it **built in**. In all of them, the buttons are the same commands, **in disguise**.

what you see

dedicated software
GitHub Desktop, GitKraken

code editors
VS Code, WebStorm

↓ each button runs a command

what happens

`git status` `git add` `git commit` `git log` `git diff` `git push`

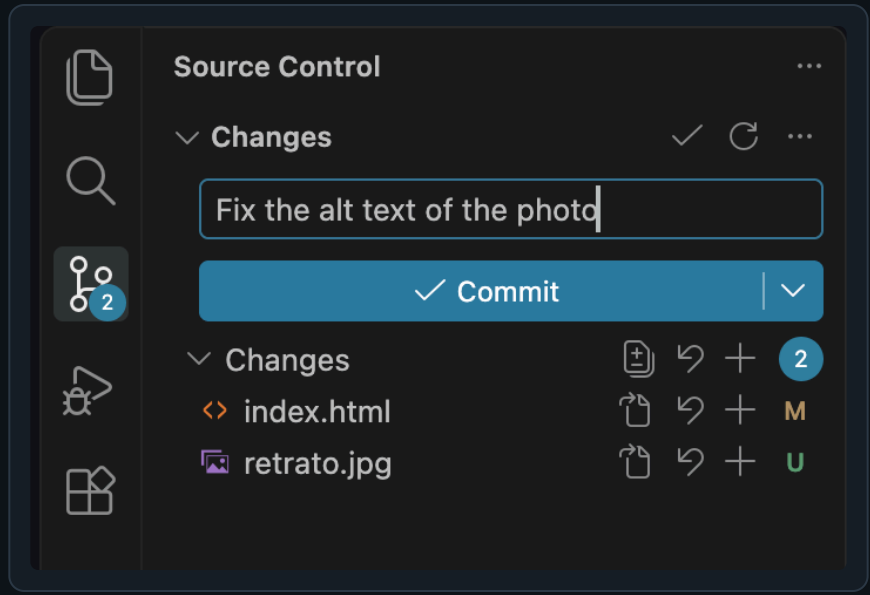
The commands used in the terminal apply to any of these tools.

// vs code

The same in VS Code

The **Source Control** panel (`Ctrl` + `Shift` + `G`) does the same as the commands, with buttons.

- The **+** next to a file does `git add`.
- The message box and the **Commit** button do `git commit -m`.
- The letters **U** (untracked) and **M** (modified) are the `git status` states.

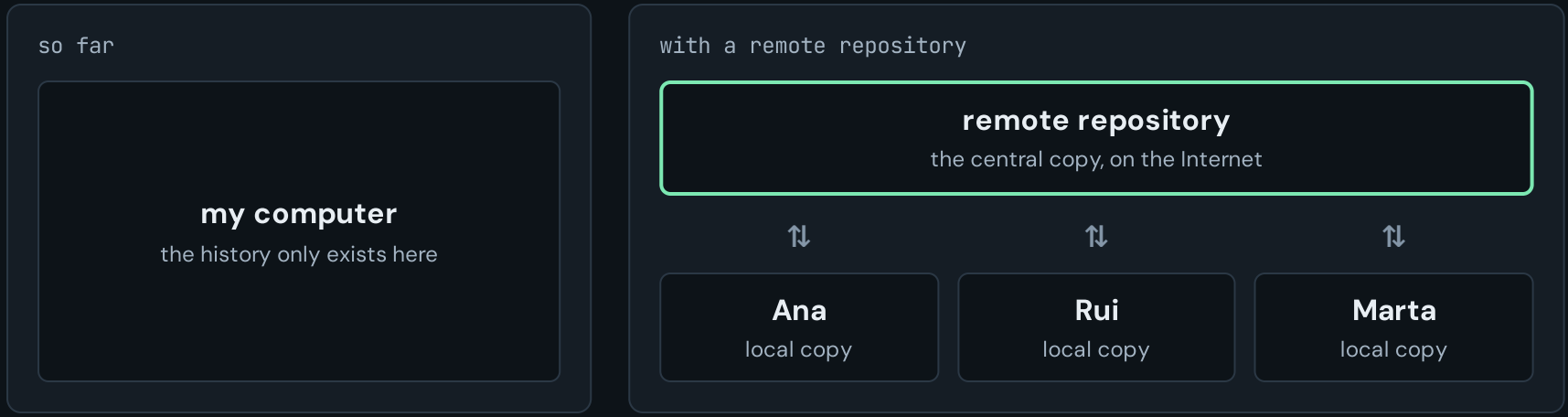


The Source Control panel, with two files not yet recorded.

// remote

So far, everything was local

The commits made so far are only on each person's computer. To work as a **team**, the project also has to be in a **remote repository**, a central repository where everyone sends their commits and gets the commits of the others.



// github

GitHub and the alternatives

GitHub is the most widely used service for keeping remote repositories. Besides keeping the project, it shows the history in the browser, brings several people together in the same repository and publishes websites, with the GitHub Pages of the next lesson.

GitHub

used in this course

the most used; owned by Microsoft since 2018

GitLab

can also be installed on your own server

Codeberg

non-profit, run from Europe

Bitbucket

by Atlassian, widely used in companies

They all keep Git repositories, so the commands are the same in any of them.

// git and github

Git and GitHub are not the same

Git works on the computer and **GitHub** keeps a copy of the repository on the Internet. `git push` connects the two.

Git

local

what it is	a program
where it is	on the computer
Internet	not needed
used to	record the history
owned by	free software

`git push` →

sends the commits from local to remote

GitHub

remote

what it is	an online service
where it is	on a server, on the Internet
Internet	needed
used to	keep, share and publish
owned by	Microsoft

// remote

Linking a local repository to a remote

Clone a repository that already exists on GitHub. `git clone` creates the folder, with the whole history, already linked to the remote.

```
git clone https://github.com/ana-silva/meu-site.git
```

Link a local repository to an empty repository created on GitHub. The `-u` is only needed for the first upload.

```
git remote add origin https://github.com/ana-silva/meu-site.git
git push -u origin main
```

In VS Code, **Publish Branch** creates the repository on GitHub, links it and does the first upload all at once.

// push

Sending commits with git push

git push sends to the remote repository, on GitHub, the commits that are still only on the computer.

Only **commits** are sent. Changes not yet saved, or in staging, stay on the computer.

The first upload is done with **Publish Branch** in VS Code, which creates the repository on GitHub. After that, `git push` is enough.

```
git push
```

```
To https://github.com/ana-silva/meu-site.git
   c07a5e3..f3b81d6  main -> main
```

GitHub had the history up to `c07a5e3`. With the push, it now has it up to `f3b81d6`.

// push

What push does

Commit `f3b81d6` was made on the computer, but GitHub does not have it yet. **Push** copies to the remote the commits that only exist locally.



After the push, the two histories are the **same**. The earlier commits are not sent again.

// readme

The repository’s README

`README.md` is the repository’s front page, which GitHub shows below the list of files. It is written in **Markdown**, where `#` marks a heading.

README.md

```
# Mini-site

Ana Silva's site, with an "about me" page and a contact form.
```

ACCESSIBILITY As in HTML, headings follow the order (`#` , then `##` , without skipping levels) and images have alternative text, inside the square brackets of `![…](foto.jpg)` .

03 · activity

Activity 3

The project on GitHub

14 min · individual

// activity 3

The project on GitHub

01 In the Source Control panel, click **Publish Branch**, choose a **public** repository and authorise VS Code in the browser.

02 Open the repository on GitHub and check the files and the commits.

03 Create `README.md` with a heading (`#`), a sentence about the site and the author's name.

04 Commit the README and send it with `git push`, in the terminal.

05 Reload the repository page on GitHub and look at the README.

> The repository has to be public, for GitHub Pages in the next lesson.

// cheat sheet

Commands from this lesson

COMMAND	WHAT IT IS FOR
`git --version`	checks the installation
`git config`	sets your name and email
`git init`	creates the repository

COMMAND	WHAT IT IS FOR
`git status`	shows the state
`git diff`	shows what changed
`git add .`	adds all to staging
`git commit -m "…"`	records a commit
`git log --oneline`	shows the history
`git push`	sends the commits

The first upload to GitHub is done with **Publish Branch** in VS Code.

UTPORTO
Universidade Técnica do Porto

NORTE 2030
Programa Regional do Norte

PORTUGAL 2030

Cofinanciado pela União Europeia

// end

Next lesson

Publish the site on **GitHub Pages** and test it with the keyboard and a screen reader.

Until then, bring the repository on GitHub with at least **3 commits**.

CTeSP DTAM
Tecnologias Web

Lecturer
Gonçalo Reais